

Lista Teórica 06 — Gabarito

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Aula 06 — Pré-processamento e *Pipelines*

Exercício 1 — revisando o notebook de um colega. Um colega lhe manda o notebook dele na véspera de entregar o relatório e pede uma olhada. Ele leu a Aula 06 na diagonal, e o resultado é um pouco de cada coisa: em alguns trechos fez exatamente o certo; em outros cometeu um deslize que não muda nada; e em pelo menos um está prestes a reportar um número que não existe.

Para separar os três casos você tem uma pergunta só, e ela não é “quanto essa etapa aprendeu dos dados”. É: **essa etapa olhou o Y ?**

Diga, para cada trecho, se o vazamento é **grave**, **leve**, ou se não há vazamento nenhum — e justifique em uma linha, aplicando a pergunta.

- (a) Ele padronizou as covariáveis com a média e o desvio do banco inteiro, e só depois separou treino e teste.
- (b) De 10 000 covariáveis, ficou com as 50 mais correlacionadas com Y — correlação calculada no banco inteiro — e rodou a validação cruzada só nessas 50.
- (c) Ajustou um PCA no banco inteiro, guardou as 20 primeiras componentes, e aí separou treino e teste.
- (d) Preencheu os faltantes com a mediana da própria coluna, calculada no banco inteiro.
- (e) O banco tem uma linha por consulta, e o mesmo paciente aparece em várias delas. Ele rodou KFold sobre as linhas.
- (f) Pôs o `StandardScaler` dentro de um `Pipeline`, e passou o *pipeline* inteiro para o `GridSearchCV`.

Solução

Vale agrupar por veredito, porque dentro de cada grupo o argumento é o mesmo: **(a), (c) e (d) são leves; (b) e (e) são graves; e (f) não é vazamento** — é justamente a conduta correta.

Os três leves. Todos mexem só em X . O `StandardScaler` de (a) calcula duas estatísticas por coluna, a média e o desvio; a imputação de (d) calcula uma, a mediana; e o PCA de (c) procura direções de máxima variância. Em nenhum dos três o Y entra na conta, e é isso que os salva: sem olhar a resposta, não existe caminho por onde fabricar acerto. Trocar o treino pelo banco inteiro desloca essas estatísticas por algo da ordem de $1/\sqrt{n}$, e desloca igual para todas as observações, independentemente do y de cada uma.

O (c) é o que mais engana, e vale parar nele. Ajustar o PCA no banco todo *de fato* usa informação do teste — as componentes saem outras. Mas “usar informação” e “inflar o número que se vai reportar” são coisas diferentes, e sem olhar o Y só a primeira acontece. A Aula E2 mediu quatro configurações com y de ruído puro, e em nenhuma delas o R^2 subiu.

Os dois graves, e por motivos distintos. No (b), a escolha das 50 colunas foi feita *pela correlação com Y* — inclusive com a parte do Y que está nas dobras de validação. Quando a validação cruzada roda depois, cada dobra encontra covariáveis que ela própria

ajudou a escolher, e reencontra ali o seu próprio ruído. Com 10 000 colunas disponíveis, o acaso tem espaço de sobra: a Aula 06 mede $R^2 = +0,40$ num caso em que a resposta é ruído puro e o valor verdadeiro é zero.

Já o (e) não é sobre pré-processamento nenhum. É a *mesma unidade* dos dois lados da divisão: consultas do mesmo paciente, umas no treino e outras na validação. O modelo não precisa aprender nada sobre medicina — basta reconhecer o paciente, que ele já viu.

E o (f), que é o que se deve fazer: dentro do *pipeline*, a padronização é reajustada em cada dobra, com a média e o desvio de quem está no treino *daquela* dobra. Nada vaza, e não custou esforço nenhum.

O par que vale guardar é este: **o Pipeline resolve (a), (b), (c) e (d)** — basta pôr cada etapa dentro dele, a seleção de variáveis inclusive —, **mas não faz nada pelo (e)**. Ele controla *quando* cada etapa é ajustada, não *como* as linhas são repartidas, e não tem como saber que duas delas são o mesmo paciente. Para o (e) a ferramenta é o `GroupKFold`, com o identificador do paciente em `groups`. São duas perguntas independentes, e você precisa das duas na cabeça: *a etapa olha o Y?* e *a divisão respeita a unidade?*

Exercício 2 — a receita inteira, num objeto só. Você chega a um banco e recebe a tarefa de sempre: prever inadimplência. O arquivo tem seis colunas. Três são numéricas — `idade`, `renda` e `saldo` —, e a `renda` vem com buracos, porque nem todo cliente informa. Duas são texto, `estado` e `profissao`, e essas vieram completas. A sexta é a resposta, `inadimplente`, que vale 0 ou 1.

O que você quer fazer com elas é o que a aula descreveu: preencher os faltantes das numéricas pela mediana, padronizá-las, transformar as duas categóricas em indicadoras, e ajustar uma regressão logística com penalização ℓ_2 , escolhendo o C por validação cruzada de cinco dobras. E quer tudo isso sem que o número que você vai apresentar no fim saia otimista.

- Escreva o código, com `ColumnTransformer`, `Pipeline` e `GridSearchCV`.
- Se a grade de C tem seis valores, quantas vezes a mediana da `renda` acaba sendo calculada ao longo de toda a busca? Justifique, e compare com o que aconteceria se você a imputasse uma vez só, no começo.
- Por que a ordem tem de ser imputar *e depois* padronizar, e não o contrário?

Solução

(a) As colunas pedem tratamentos diferentes, então o `ColumnTransformer` abre dois ramos — e o ramo numérico é, ele próprio, um *pipeline* de duas etapas:

```
1 import numpy as np
2 import sklearn.model_selection as skm
3 from sklearn.compose import ColumnTransformer
4 from sklearn.impute import SimpleImputer
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.pipeline import Pipeline
7 from sklearn.preprocessing import OneHotEncoder, StandardScaler
8
9 numericas = ["idade", "renda", "saldo"]
10 categoricas = ["estado", "profissao"]
11
12 prep = ColumnTransformer([
13     ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
14     ("escala", StandardScaler())])), numericas),
```

```

15     ("cat", OneHotEncoder(handle_unknown="ignore"), categoricas),
16 ])
17
18 tubo = Pipeline([("prep", prep),
19                  ("modelo", LogisticRegression(max_iter=2000))])
20
21 busca = skm.GridSearchCV(
22     tubo,
23     {"modelo__C": np.logspace(-3, 2, 6)},
24     cv=skm.StratifiedKFold(5, shuffle=True, random_state=0),
25     scoring="roc_auc",
26 ).fit(X, y)

```

Três escolhas do código merecem uma palavra. O `handle_unknown="ignore"` evita que o ajuste quebre quando uma profissão rara aparece só na validação e o codificador a vê pela primeira vez — é o tipo de erro que não acontece no seu notebook e acontece em produção, seis meses depois. O `StratifiedKFold` mantém a proporção de inadimplentes em cada dobra: em classificação ele já é o padrão do `GridSearchCV`, mas escrevê-lo deixa o `shuffle` sob o seu controle, e não sob o de quem ordenou o arquivo (Exercício 3 da Lista prática 03). E a penalização ℓ_2 que o enunciado pede **não precisa ser escrita**: é o padrão da `LogisticRegression`. Escrever `penalty="l2"` não está errado, mas o argumento foi depreciado na versão 1.8 do `scikit-learn` e desaparece na 1.10 — omiti-lo funciona em qualquer versão.

(b) Trinta vezes: seis valores de C vezes cinco dobras. Para cada combinação, o `GridSearchCV` chama o `fit` da *pipeline inteira* usando só os 4/5 de treino daquela dobra, e ali dentro o `SimpleImputer` recalcula a mediana do zero. (Com `refit=True`, que é o padrão, ainda há um 31º ajuste no conjunto completo ao final, para construir o `best_estimator_`.)

Comparado com imputar uma vez só no começo, isso é trinta vezes mais trabalho para chegar à mesma coluna preenchida — ou quase a mesma, e é justamente o *quase* que interessa. A mediana muda de dobra para dobra, e essa variação faz parte do que você está tentando medir: o desempenho não é do modelo sozinho, é do *procedimento* que imputa, padroniza e ajusta. Imputar antes tira essa variação da conta e usa, em cada dobra, uma mediana que as observações de validação ajudaram a calcular.

Pelo critério do Exercício 1 isso é um vazamento *leve* — a mediana não olha o Y —, e o efeito numérico seria pequeno. A questão é que corrigi-lo custou zero: já estava escrito.

(c) Pelo motivo prosaico primeiro: se você padronizar antes, o `StandardScaler` encontra `NaN` na coluna `renda` e a média e o desvio saem `NaN` também, contaminando a coluna inteira. Na melhor das hipóteses o código quebra na sua frente; na pior ele segue em silêncio com uma coluna de `NaN`.

E há o motivo conceitual, que sobrevive mesmo se a implementação fosse esperta o bastante para ignorar os faltantes. Imputar depois de padronizar significa inserir a mediana *da escala já padronizada*: você acabou de deixar a coluna com média 0 e desvio 1, e aí acrescenta um punhado de valores que desloca as duas coisas. A padronização que você fez deixa de valer para os dados que efetivamente chegam ao modelo. Imputando primeiro, a coluna chega completa ao `StandardScaler`, e a média e o desvio que ele calcula são os dos números que o modelo vai de fato ver.